RAG(Retrieval Augmented Generation) 소개 및 실습 (환경 셋팅 - uv 기반)

배 경 만(kyoungman.bae@etri.re.kr)
시각지능연구실 / 한국전자통신연구원

Chapter

1

환경 설정

가상환경이란 무엇인가?

아파트 비유로 쉽게 이해해 봅시다.

아파트 한 동을 상상해 보세요

건물 전체(Python 설치)는 하나입니다.
그런데 각 **호수(가상환경)**는 독립된 공간입니다.
101호 주민이 가구를 바꿔도 201호는 전혀 영향받지 않죠.

가상환경도 마찬가지입니다. Python은 하나 설치해두고,
프로젝트마다 독립된 패키지 공간을 만들어 쓰는 것입니다.

가상환경 없이

requests	2.28
numpy	1.23
torch	2.0

⚠ 모든 프로젝트가 공유

vs

프로젝트 A 환경

requests	2.28
numpy	1.23

프로젝트 B 환경

requests	2.31
torch	2.3

핵심 한 줄 요약: 가상환경은 "프로젝트별 전용 패키지 창고"입니다.
창고가 다르니 서로 건드릴 일이 없고, 언제든 깨끗하게 정리할 수 있습니다.

Copyright© 2021 by ETRI

환경 설정

자주 묻는 질문

Q & A

Q. 가상환경을 삭제하면 소스코드도 사라지나요 ?

아닙니다 . 소스코드 (.py 파일) 와 가상환경 (.venv 폴더) 은 완전히 별개입니다 . .venv 를 지워도 코드는 그대로이고 , 환경만 다시 만들면 됩니다 .

Q. Python 이 여러 개 설치되는 건가요 ?

기본적으로 Python 본체는 하나입니다 . 가상환경은 그 Python 을 공유하면서 패키지 공간만 분리합니다 . uv 는 Python 버전도 별도 관리할 수 있습니다 .

Q. 매번 activate(켜기) 를 해야 하나요 ?

VSCode·PyCharm 같은 편집기는 .venv 를 자동 감지해서 켜줍니다 . 터미널에서 직접 쓸 때만 source .venv/bin/activate 를 입력하면 됩니다 .

Q. conda 는 왜 라이선스 문제가 있나요 ?

conda 자체 (오픈소스) 가 아니라 Anaconda 배포판의 'defaults' 채널이 상업적 이용 시 유료입니다 . conda-forge 채널은 무료이지만 , 처음부터 uv 를 쓰면 이 고민이 없습니다 .

Copyright© 2021 by ETRI

- 가상 환경 비교

항목	conda	venv	uv
라이선스	⚠️ Anaconda 상업적 이용 유료	✅ 완전 무료 (Python 내장)	✅ 완전 무료 (MIT)
설치 속도	🐢 매우 느림	🚶 보통	⚡ **10~100배 빠름**
의존성 해결	보통	기본 pip	✅ **자동 충돌 해결**
CUDA/torch 지원	좋음	pip 수동 지정	✅ **인덱스 URL 지정 가능**
Python 버전 관리	가능	❌ 불가	✅ **자동 설치 가능**
lock 파일	❌	❌	✅ **재현성 보장**
용량	크고 무거움	가벼움	⚡ **매우 가벼움**

Copyright© 2021 by ETRI

환경 설정

- 가상 환경 비교(명령어)

환경 만들기

```
# conda
conda create -n 이름 python=3.10

# venv
python -m venv .venv

# uv
uv venv --python 3.10 .venv
```

환경 켜기

```
# conda
conda activate 이름

# venv / uv (Linux·Mac)
source .venv/bin/activate

# venv / uv (Windows)
.venv\Scripts\activate
```

패키지 설치

```
# conda
conda install 패키지명

# venv
pip install 패키지명

# uv (10~100배 빠름)
uv pip install 패키지명
```

환경 끄기

```
# 세 도구 모두 동일
deactivate

# 환경 삭제 (uv/venv)
rm -rf .venv

# 환경 삭제 (conda)
conda env remove -n 이름
```

환경 목록 보기

```
# conda
conda env list

# uv / venv
# 전용 명령 없음
find ~ -name ".venv" -maxdepth 4
```

환경 저장·복원

```
# conda
conda env export > env.yml
conda env create -f env.yml

# uv / venv
pip freeze > requirements.txt
uv pip install -r
requirements.txt
```

Copyright© 2021 by ETRI

가상환경 설정

- 가상 환경 비교(conda 명령어 vs uv 명령어)

작업	conda	uv
환경 생성	`conda create -n rag_env python=3.10`	`uv venv --python 3.10 .venv`
환경 활성화	`conda activate rag_env`	`source .venv/bin/activate`
패키지 설치	`conda install pandas`	`uv pip install pandas`
pip 설치	`pip install gradio`	`uv pip install gradio`
환경 파일로 생성	`conda env create -f env.yml`	`uv pip install -r requirements.txt`
환경 내보내기	`conda env export > env.yml`	`uv pip freeze > requirements.txt`
환경 삭제	`conda env remove -n rag_env`	`rm -rf .venv`

Copyright© 2021 by ETRI

- uv 환경 사용

Copyright© 2021 by ETRI

- uv 환경 사용

Q1. 프로젝트 폴더로 가야만 활성화 가능한가?

NO 경로만 맞으면 어디서든 활성화 가능

```
# 프로젝트 폴더 안에서
source .venv/bin/activate
```

```
# 어느 위치에서든 절대경로로
source ~/projects/rag-tutorial/.venv/bin/activate
```

✓ 둘 다 동작 — 경로가 핵심, 위치는 무관

Q2. 하나의 프로젝트에 여러 환경을 만들 수 있나?

YES 단, .venv 이름을 직접 바꿔서 수동 관리

```
rag-tutorial/
    ├── .venv-py310-cu118/ ← 환경 A
    ├── .venv-py310-cu124/ ← 환경 B
    ├── .venv-py311/ ← 환경 C
    └── (소스코드들)
```

⚠ uv가 자동 인식하는 건 .venv 하나뿐

```
# 환경 A 활성화
uv venv --python 3.10 .venv-py310-cu118
source .venv-py310-cu118/bin/activate

# 환경 B로 전환
deactivate
source .venv-py310-cu124/bin/activate
```

Q3. 환경만 다르게 하려면 프로젝트 폴더를 복사해야 하나?

NO 소스코드는 공유하고 환경(.venv-*)만 여러 개 두면 됨

방법	폴더 복사 (비권장)	.venv 분리 (권장)
소스코드 중복	있음	없음
git 관리	폴더마다 따로	하나로 통합
전환 방법	cd 폴더 이동	source .venv-X/bin/activate
.gitignore	.venv 각각 추가	.venv* 한 줄로 끝

Copyright© 2021 by ETRI

환경 설정

- setup_rag_uv.sh
  - 환경 확인 ➔ uv 설치

```
(base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv$ ./setup_rag_uv.sh

═══════════════════════════════════════
 STEP 0: 시스템 환경 사전 확인
═══════════════════════════════════════
[INFO]  ※ uv 가상환경은 base Python과 완전히 격리됩니다.
[INFO]    base에 어떤 Python이 설치돼 있어도 충돌하지 않습니다.

[INFO]  OS: Ubuntu 24.04.2 LTS
[INFO]  시스템에 설치된 Python 버전 감지 중 ...
[OK]    사용할 Python: 3.12 (Python 3.12.8)
[WARN]  권장 버전은 3.10입니다. 다른 버전을 원하면: bash setup_rag_uv.sh 3.10
[OK]    GPU 감지: NVIDIA GeForce RTX 3090
[INFO]  드라이버: 550.144.03  |  CUDA: |
[INFO]  PyTorch wheel 선택: cpu  (index: https://download.pytorch.org/whl/cpu)
[OK]    Java 감지: openjdk version "21.0.10" 2026-01-20

═══════════════════════════════════════
 STEP 1: uv 설치
═══════════════════════════════════════
[INFO]  uv 설치 중 ...
downloading uv 0.11.7 x86_64-unknown-linux-gnu
installing to /home/qa/.local/bin
  uv
  uvx
everything's installed!
[INFO]  .bashrc에 이미 .local/bin 경로가 등록되어 있습니다.
[OK]    uv 설치 완료: uv 0.11.7 (x86_64-unknown-linux-gnu)
```

PATH를 소스상에서 자동 추가 ➔ source 실행은 필요

Copyright© 2021 by ETRI

환경 설정

- setup_rag_uv.sh
  - 실행 소스 클론 ➔ 가상환경 생성 및 활성화 ➔ torch 설치

```
═══════════════════════════════════
 STEP 2: rag-tutorial 저장소 클론
═══════════════════════════════════
[INFO]  저장소 클론 중 : https://github.com/KyungmanBae/rag-tutorial.git
Cloning into 'rag-tutorial'...
remote: Enumerating objects: 133, done.
remote: Counting objects: 100% (133/133), done.
remote: Compressing objects: 100% (122/122), done.
remote: Total 133 (delta 58), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (133/133), 8.57 MiB | 9.71 MiB/s, done.
Resolving deltas: 100% (58/58), done.
[OK]    클론 완료 : /home/qa/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial
```

클론된 폴더

```
═══════════════════════════════════
 STEP 3: Python 3.12 가상환경 생성
═══════════════════════════════════
Using CPython 3.12.8 interpreter at: /home/qa/miniforge3/bin/python3
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate
[OK]    가상환경 생성 완료 : /home/qa/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial/.venv  (Python 3.12)
[OK]    가상환경 활성화 : /home/qa/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial/.venv/bin/python  |  Python Python 3.12.8
```

환경 생성

```
═══════════════════════════════════
 STEP 4: PyTorch 설치 (cu124)
═══════════════════════════════════
[INFO]  설치 중 : torch torchvision torchaudio (cu124)
Resolved 28 packages in 3.63s
Prepared 28 packages in 31.29s
Installed 28 packages in 215ms
 + filelock==3.25.2
 + fsspec==2026.2.0
 + jinja2==3.1.6
 + markupsafe==3.0.3
```

cuda 버전을 확인 후 설치

Copyright© 2021 by ETRI

환경 설정

- setup_rag_uv.sh
  - 주요 패키지 설치

```
STEP 5: RAG 핵심 패키지 설치

[INFO]  [1/4] LLM 패키지 설치 중 ...
Resolved 51 packages in 139ms
Installed 26 packages in 67ms
 + accelerate==1.13.0
 + annotated-doc==0.0.4
 + anyio==4.13.0
 + bitsandbytes==0.49.2
 + certifi==2026.4.22
 + click==8.3.3
 + h11==0.16.0
 + hf-xet==1.4.3
 + httpcore==1.0.9
 + httpx==0.28.1
 + huggingface-hub==1.12.0
 + idna==3.13
 + markdown-it-py==4.0.0
 + mdurl==0.1.2
 + packaging==26.2
 + psutil==7.2.2
 + pygments==2.20.0
 + pyyaml==6.0.3
 + regex==2026.4.4
 + rich==15.0.0
 + safetensors==0.7.0
 + shellingham==1.5.4
 + tokenizers==0.22.2
 + tqdm==4.67.3
 + transformers==5.6.2
 + typer==0.25.0
[OK]    LLM 패키지 설치 완료
[INFO]  [2/4] 검색기 패키지 설치 중 ...
Resolved 73 packages in 68ms
Installed 25 packages in 51ms
 + aiohappyeyeballs==2.6.1
 + aiohttp==3.13.5
```

```
 + faiss-cpu==1.13.2
[OK]    faiss-cpu 설치 완료 (GPU 가속 없음)
[INFO]  [4/4] Gradio 설치 중 ...
Resolved 48 packages in 73ms
Installed 19 packages in 65ms
 + annotated-types==0.7.0
 + brotli==1.2.0
 + fastapi==0.136.1
 + gradio==6.13.0
 + gradio-client==2.5.0
 + groovy==0.1.2
 + hf-gradio==0.4.1
 + orjson==3.11.8
 + pydantic==2.13.3
 + pydantic-core==2.46.3
 + pydub==0.25.1
 + python-multipart==0.0.26
 + pytz==2026.1.post1
 + safehttpx==0.1.7
 + semantic-version==2.10.0
 + starlette==1.0.0
 + tomlkit==0.14.0
 + typing-inspection==0.4.2
 + uvicorn==0.46.0
[OK]    Gradio 설치 완료

STEP 6: 한국어 NLP 패키지 설치 (KoNLPy)

Resolved 5 packages in 21ms
Installed 3 packages in 18ms
 + jpype1==1.7.0
 + konlpy==0.6.0
 + lxml==6.1.0
[OK]    KoNLPy 설치 완료
```

Copyright© 2021 by ETRI

환경 설정

- setup_rag_uv.sh
  - KorFactScore 클론 및 패키지 설치 확인

```
STEP 7: KorFactScore 저장소 클론

Cloning into 'KorFactScore'...
remote: Enumerating objects: 84, done.
remote: Counting objects: 100% (84/84), done.
remote: Compressing objects: 100% (56/56), done.
remote: Total 84 (delta 29), reused 76 (delta 22), p
Receiving objects: 100% (84/84), 161.10 KiB | 8.05 M
Resolving deltas: 100% (29/29), done.
[OK]    KorFactScore 클론 완료
[INFO]  KorFactScore requirements.txt 설치 중 ...
Resolved 134 packages in 133ms
Installed 68 packages in 265ms
 + beautifulsoup4==4.14.3
 + bidict==0.23.1
 + blis==1.3.3
 + bs4==0.0.2
 + catalogue==2.0.10
 + cffi==2.0.0
 + cloudpathlib==0.23.0
 + cmudict==1.1.3
 + confection==1.3.3
 + cryptography==47.0.0
 + cymem==2.0.13
 + distance==0.1.3
 + distro==1.9.0
 + emoji==1.2.0
 + et-xmlfile==2.0.0
 + google-ai-generativelanguage==0.6.15
 + google-api-core==2.30.3
 + google-api-python-client==2.194.0
 + google-auth==2.49.2
```

```
STEP 8: 설치 검증

  [OK]  Python 버전 : 3.12.8
  [OK]  torch 버전 : 2.6.0+cu124
  [OK]  CUDA 버전 (torch): 12.4
  [OK]  GPU 사용 가능 : YES
  [OK]  GPU 이름 : NVIDIA GeForce RTX 3090
  [OK]  GPU VRAM: 23.7 GB
  [OK]  bitsandbytes: 0.49.2
  [OK]  transformers: 5.6.2
  [OK]  huggingface_hub: 1.12.0
  [OK]  sentence-transformers: 5.4.1
  [OK]  faiss: ver 1.13.2  |  GPU 지원 : 0 개
  [OK]  rank_bm25: 설치됨
  [OK]  datasets: 4.8.4
  [OK]  gradio: 6.13.0
  [OK]  konlpy (Okt 형태소 분석): 설치됨  |  테스트 : ['안녕하세요', 'RAG', '실습',

  검증 결과 : 15 성공 / 0 실패

설치 완료!

[INFO]  사용된 Python 버전 : 3.12
[INFO]  가상환경 위치 : /home/qa/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial/.venv

  다음 명령으로 실습을 시작하세요 :

    cd rag-tutorial
    source .venv/bin/activate

    python llm_model_manager.py   # Step1: LLM 기반 생성
    python external_rag.py        # Step2: 외부 검색 + LLM 생성
    python internal_rag.py        # Step3: 내부 검색 + LLM 생성
    python korquad_rag.py         # KorQuAD 검색기 학습
    python korfactscore_lab.py --korfs_path ./KorFactScore
```

Copyright© 2021 by ETRI

환경 설정

- 가상환경 활성화

```
(base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv$ ll
total 32
drwxr-xr-x  3 qa qa  4096 Apr 27 07:47 ./
drwxr-xr-x 17 qa qa  4096 Apr 27 06:37 ../
drwxrwxr-x  8 qa qa  4096 Apr 27 07:47 rag-tutorial/
-rwxr--r--  1 qa qa 20199 Apr 27 07:46 setup_rag_uv.sh*
(base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv$ cd rag-tutorial/
(base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$ ll
total 10800
drwxrwxr-x 8 qa qa    4096 Apr 27 07:47  ./
drwxr-xr-x 3 qa qa    4096 Apr 27 07:47  ../
-rw-rw-r-- 1 qa qa   29465 Apr 27 07:47  build_kowiki_db.py
drwxrwxr-x 2 qa qa    4096 Apr 27 07:47  colab/
drwxrwxr-x 2 qa qa    4096 Apr 27 07:47  data/
drwxrwxr-x 2 qa qa    4096 Apr 27 07:47  env/
-rw-rw-r-- 1 qa qa   27528 Apr 27 07:47  external_rag.py
drwxrwxr-x 8 qa qa    4096 Apr 27 07:47  .git/
-rw-rw-r-- 1 qa qa    5081 Apr 27 07:47  .gitignore
-rw-rw-r-- 1 qa qa   46045 Apr 27 07:47  internal_rag.py
drwxrwxr-x 9 qa qa    4096 Apr 27 07:47  KorFactScore/
-rw-rw-r-- 1 qa qa   45468 Apr 27 07:47  korfactscore_lab.py
-rw-rw-r-- 1 qa qa   66600 Apr 27 07:47  korquad_rag.py
-rw-rw-r-- 1 qa qa    1889 Apr 27 07:47  LICENSE.md
-rw-rw-r-- 1 qa qa   30353 Apr 27 07:47  llm_model_manager.py
-rw-rw-r-- 1 qa qa 3964251 Apr 27 07:47 'RAG(Retrieval Augmented Generation) 실 습  - colab.pdf'
-rw-rw-r-- 1 qa qa 6757420 Apr 27 07:47 'RAG(Retrieval Augmented Generation) 소 개  및  실 습 .pdf'
-rw-rw-r-- 1 qa qa   18724 Apr 27 07:47  README.md
-rw-rw-r-- 1 qa qa    7511 Apr 27 07:47  TERMS.md
drwxrwxr-x 5 qa qa    4096 Apr 27 07:47  .venv/
(base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$ source .venv/bin/activate
(.venv) (base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$ .venv/bin/python -c "import torch; print(torch.__version__)"
2.6.0+cu124
(.venv) (base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$ .venv/bin/pip list
| grep -E "torch|transformers|gradio|faiss"
-bash: .venv/bin/pip: No such file or directory
(.venv) (base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$ uv pip list
Command 'uv' not found, but can be installed with:
sudo snap install astral-uv
(.venv) (base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$ .venv/bin/python -m
 pip list 2>/dev/null || uv pip list
Command 'uv' not found, but can be installed with:
sudo snap install astral-uv
(.venv) (base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$ source ~/.bashrc
```

Github에서 클론한 폴더

폴더 내부에 환경 생성

설치된 패키지 확인

PATH 문제➔ 반영

~/.local/bin/uv 가 PATH에 없으면 못 찾음

Copyright© 2021 by ETRI

환경 설정

- 가상환경 활성화
  - 가상환경의 상태 확인
  - 실행 python 경로
  - 가상 환경의 환경 변수 확인

```
(base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$ echo $VIRTUAL_ENV

(base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$ which python
/home/qa/miniforge3/bin/python
(base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$
(base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$ source .venv/bin/activate
(.venv) (base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$
(.venv) (base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$ echo $VIRTUAL_ENV
/home/qa/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial/.venv
(.venv) (base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$
(.venv) (base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$ which python
/home/qa/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial/.venv/bin/python
(.venv) (base) qa@qa:~/kmbae/aiacademy/rag-tutorial-uv/rag-tutorial$
```

Copyright© 2021 by ETRI